AI ENGINEER · PROJECT PORTFOLIO

Ideas into AI Systems.

컴퓨터 비전부터 생성형 AI와 RAG까지, 문제를 정의하고 실제로 작동하는 서비스로 구현한 프로젝트를 소개합니다.

PROJECTS	FOCUS	PORTFOLIO
07	CV · LLM · GenAI	2026

AI ENG

SELECTED WORK

Project Index

01

설비 고장 예측 기반 정비 의사결정 지원 시스템 ↗

Data · ML

정비 담당자가 오늘 먼저 확인할 설비와 필요한 부품을 한눈에 보고, 점검·발주·정비 일정을 결정하도록 돕는 대시보드입니다.

02

소상공인 맞춤 가게 광고 이미지 생성 ↗

Generative AI

상품 사진과 매장 정보만 입력하면 상품의 정체성을 보존한 광고 이미지부터 업종별 카피와 레이아웃까지 자동으로 완성하는 소상공인 맞춤형 AI 광고 제작 서비스입니다.

03

RFP 기반 RAG 질의응답 시스템 ↗

LLM · RAG

복잡한 RFP 문서에서 요구사항, 일정, 평가 기준을 빠르게 찾고 근거 기반 답변을 생성하는 RAG 기반 질의응답 시스템입니다.

04

요양보호사 자격증 자동 검증 시스템 ↗

Computer Vision

요양보호사가 Carebridge 앱에 자격증 이미지를 업로드하면 OCR로 핵심 정보를 인식하고, 국시원 API를 통해 자격증 진위 여부를 자동 확인하는 기능입니다.

05

알약 인식 및 분류 모델 ↗

Computer Vision

알약 이미지를 입력하면 이미지 속 알약의 위치를 찾고 종류를 예측하는 컴퓨터비전 기반 AI 서비스입니다.

06

딥페이크 이미지 탐지 모델 ↗

Computer Vision

사람 얼굴 이미지를 입력하면 딥페이크 여부를 판별하고, 딥페이크일 확률을 예측하는 컴퓨터비전 기반 AI 모델입니다.

07

콘크리트 균열 탐지 및 유형 분류 모델

Computer Vision

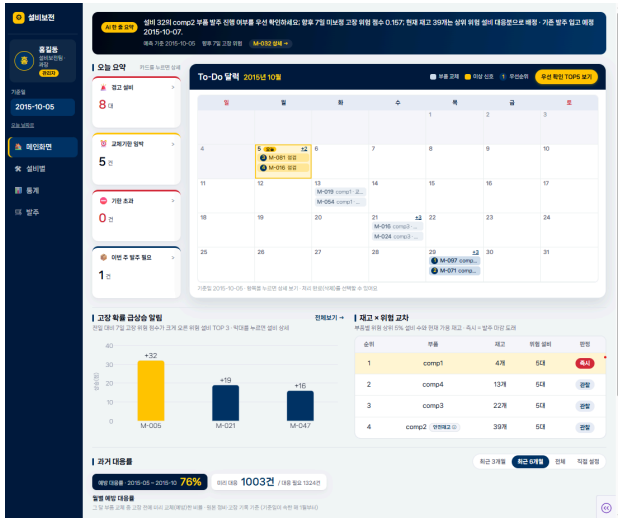
콘크리트 이미지를 입력하면 MobileNetV2 기반 분류 모델이 crack 없음, wide crack, hairline crack 중 하나로 예측하는 컴퓨터비전 모델입니다.

DATA · ML

설비 고장 예측 기반 정비 의사결정 지원 시스템

01

정비 담당자가 오늘 먼저 확인할 설비와 필요한 부품을 한눈에 보고, 점검·발주·정비 일정을 결정하도록 돕는 대시보드입니다.



관리 대상
설비 100대

센서 기록
876,100건

지원 업무
점검 · 발주 · 정비

01 · PROBLEM

문제 정의

고장 가능성을 알아도 어떤 설비부터 확인하고 부품을 언제 준비해야 하는지 바로 판단하기 어려웠습니다.

02 · APPROACH

해결 방법

오늘 할 일, 이상 원인, 부품 재고와 일정을 한 화면에서 확인하고 바로 조치를 정할 수 있게 구성했습니다.

03 · OUTCOME

결과

100대 설비 중 우선 확인할 대상을 보여주고, 점검부터 발주·정비 결정까지 이어지는 업무 흐름을 구현했습니다.

04 · LEARNING

배운 점

현장에서는 예측 수치만큼 경고 이유와 지금 해야 할 일을 쉽게 보여주는 것이 중요하다는 점을 배웠습니다.

PROCESS
서비스 흐름

01 위험 설비 확인 › 02 상세 원인 확인 › 03 재고·일정 확인 › 04 발주·정비 결정

2026.09.21 ~ 2026.09.30 · 설비·운영 데이터 정리 및 분석

Python · Pandas · scikit-learn · LightGBM · Isolation Forest · PyTorch · Dash · Plotly · Qwen3-1.7B

GitHub 저장소 열기 ↗

GENERATIVE AI · TEAM LEAD

소상공인 맞춤 가게 광고 이미지 생성

02

상품 사진과 매장 정보만 입력하면 상품의 정체성을 보존한 광고 이미지부터 업종별 카피와 레이아웃까지 자동으로 완성하는 소상공인 맞춤형 AI 광고 제작 서비스입니다.



자동 생성 범위	이미지 생성	서비스 형태
이미지·카피·레이아웃	SD + ControlNet	End-to-End

01 · PROBLEM

문제 정의

생성된 광고 제목과 보조 문구를 이미지에 자동으로 배치하는 작업을 담당했습니다. VLM을 한 번만 사용해 레이아웃을 만들었을 때 텍스트 위계, 여백, 상품과 문구 사이의 시선 흐름이 어색해 디자인 완성도가 낮고 결과가 일관되지 않는 문제가 있었습니다.

02 · APPROACH

해결 방법

논문 조사에서 VLM으로 초안 디자인을 만든 뒤 다시 한 번 검토·수정하면 성능이 향상된 사례를 확인했습니다. 하지만 이 방식을 그대로 적용해 두 번째 VLM에 수정을 요청했을 때는 레이아웃이 충분히 개선되지 않았습니다. 그래서 현재 서비스에 맞게 구조를 바꿔, 첫 번째 레이아웃의 Good Point와 Bad Point를 10개 생성하게 하고 이 구체적인 평가를 바탕으로 두 번째 레이아웃을 다시 설계하도록 구성했습니다.

03 · OUTCOME

결과

카피라이팅 문구 생성부터 1차 자동 배치, 디자인 장단점 평가, 2차 레이아웃 재설계까지 이어지는 파이프라인을 구현했습니다. 단순히 VLM을 두 번 호출했을 때보다 평가 기준이 구체화되면서 텍스트 위계와 여백, 가독성이 개선되어 훨씬 완성도 높은 광고 레이아웃을 얻을 수 있었습니다.

04 · LEARNING

배운 점

문제가 생겼을 때 논문에서 검증된 방법을 참고하는 것은 중요하지만, 방법을 그대로 적용하는 것만으로는 같은 결과를 얻을 수 없다는 점을 배웠습니다. 논문의 핵심 아이디어를 이해한 뒤 현재 데이터와 서비스 조건에 맞게 평가 기준과 처리 구조를 튜닝해야 실제 성능 개선으로 이어진다는 점을 체감했습니다.

PROCESS

서비스 흐름

01 업종 유형 선택 › 02 상품 사진·매장 정보 입력 › 03 광고 스타일 설정 › 04 이미지·카피 자동 생성 › 05 결과 저장 및 홍보 활용

2026.07 ~ 2026.07 · 팀 리더 · 생성 AI 서비스 기획, 이미지·카피·레이아웃 파이프라인 설계 및 서비스 통합

Python · Stable Diffusion · ControlNet · OpenCV · OpenAI · Streamlit · GitHub · Miro · GCP

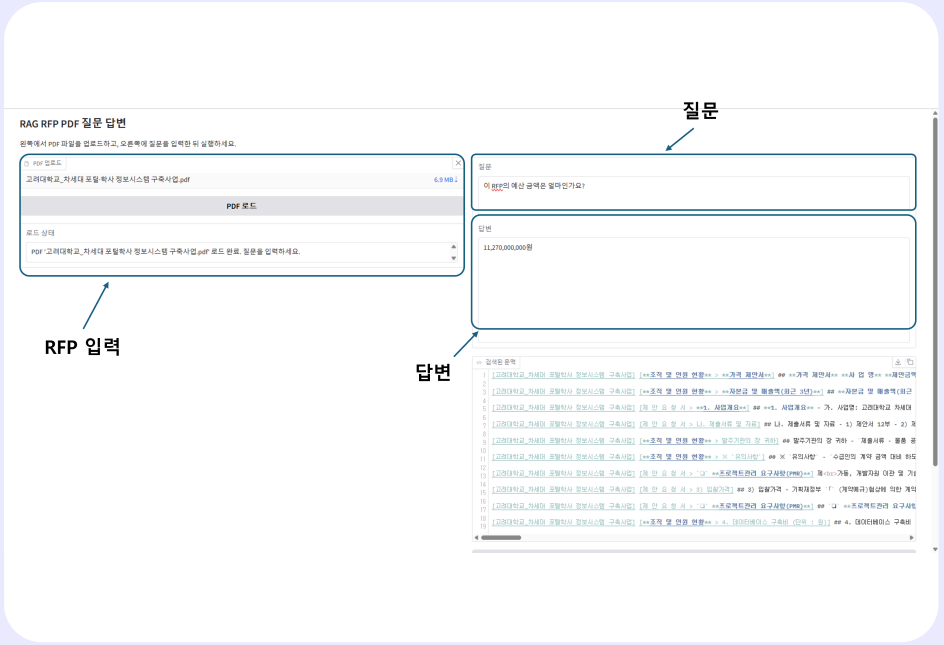
GitHub 저장소 열기 ↗

LLM · RAG · TEAM LEAD

RFP 기반 RAG 질의응답 시스템

03

복잡한 RFP 문서에서 요구사항, 일정, 평가 기준을 빠르게 찾고 근거 기반 답변을 생성하는 RAG 기반 질의응답 시스템입니다.



검색 방식
Top-k Retrieval

임베딩
bge-m3

LLM
Qwen3-8B

01 · PROBLEM

문제 정의

Hugging Face 모델의 성능을 GPT-5 mini baseline 수준까지 끌어올려야 했지만, 여러 실험을 순차적으로 수행하기에는 프로젝트 기간과 GPU 자원이 부족했습니다.

02 · APPROACH

해결 방법

팀장으로서 팀원들의 개선 아이디어를 모은 뒤, 각자 하나의 요인만 변경해 병렬로 학습하고 결과를 공유하는 실험 방식을 제안했습니다. 시간과 자원의 제약상 교호작용 분석은 제외하고, 개별 실험에서 효과가 확인된 요인을 마지막 학습에 모두 적용했습니다.

03 · OUTCOME

결과

chunk size, overlap, max token, top-k 변경이 성능 향상에 유의미하다는 결과를 도출했습니다. 네 가지 요소를 최종 파이프라인에 통합해 목표로 했던 GPT-5 mini baseline 성능을 달성했습니다.

04 · LEARNING

배운 점

제한된 자원에서는 모든 조합을 탐색하기보다 가설을 명확히 나누고 팀의 실험을 병렬화하는 것이 중요하다는 점을 배웠습니다. 또한 교호작용을 생략한 의사결정의 한계를 인지하고, 실험 조건과 근거를 투명하게 공유해야 팀의 결과를 신뢰성 있게 통합할 수 있음을 체감했습니다.

PROCESS

서비스 흐름

01 RFP 문서 등록 › 02 궁금한 내용 질문 › 03 관련 문서 내용 검색 › 04 근거가 포함된 답변 확인

2026.05.13 ~ 2026.06.04 · 팀장, RAG 파이프라인 설계, 문서 전처리, 검색 및 생성 모델 연동

Python · LangChain · ChromaDB · Qwen3-8B · bge-m3 · HuggingFace · OpenAI API · GCP · W&B

GitHub 저장소 열기 ↗

COMPUTER VISION

요양보호사 자격증 자동 검증 시스템

04

요양보호사가 Carebridge 앱에 자격증 이미지를 업로드하면 OCR로 핵심 정보를 인식하고, 국시원 API를 통해 자격증 진위 여부를 자동 확인하는 기능입니다.



OCR 엔진
PaddleOCR

검증 방식
국시원 API

연동 형태
Docker API

01 · PROBLEM

문제 정의

OCR 기능은 Python과 관련 라이브러리에 의존하지만, 백엔드 환경은 Python을 사용하지 않아 코드만 전달하면 실행 환경 차이로 연동 문제가 발생할 수 있었습니다.

02 · APPROACH

해결 방법

PaddleOCR, 전처리, 자격증 필드 파싱, 국시원 API 호출 흐름을 FastAPI 서비스로 감싸고 Docker 이미지로 패키징했습니다. 백엔드는 Python 런타임이나 OCR 라이브러리 설치 없이 컨테이너 API만 호출하도록 분리했습니다.

03 · OUTCOME

결과

요양보호사 자격증 이미지 업로드부터 OCR 인식, 국시원 API 진위 확인, JSON 결과 반환까지 이어지는 API 흐름을 구현했고, Docker 기반으로 백엔드 연동 가능한 형태로 공유했습니다.

04 · LEARNING

배운 점

기능을 구현하는 것에서 끝나지 않고 다른 부서와 환경, 호출 방식, 배포 단위를 맞춰야 실제 서비스에 연결된다는 점을 배웠습니다. 기술 구현과 커뮤니케이션이 함께 맞물릴 때 기능이 실행 가능한 형태가 된다는 점을 체감했습니다.

PROCESS

서비스 흐름

01 자격증 사진 업로드 › 02 자격 정보 자동 인식 › 03 자격증 진위 확인 › 04 검증 결과 확인

2026.06 ~ 2026.06 · PaddleOCR 기반 자격증 인식, 국시원 API 진위 확인, Docker 기반 백엔드 연동 API 구현

Python · PaddleOCR · OpenCV · FastAPI · 국시원 API · Docker

GitHub 저장소 열기 ↗

COMPUTER VISION

알약 인식 및 분류 모델

05

알약 이미지를 입력하면 이미지 속 알약의 위치를 찾고 종류를 예측하는 컴퓨터비전 기반 AI 서비스입니다.



파이프라인
탐지 + 분류

출력 결과
bbox + 라벨

서비스 형태
Gradio API

01 · PROBLEM

문제 정의

단순 classifier 구조는 unknown class를 분류하기 어렵고, 신규 class가 추가될 때마다 추가 학습이 필요하다는 운영상 한계가 있었습니다.

02 · APPROACH

해결 방법

기존 분류 head를 embedding head로 변경해 이미지 특징을 임베딩 공간에서 비교하고, 새로운 class 확장에 더 유연하게 대응할 수 있도록 구조를 개선했습니다.

03 · OUTCOME

결과

구조를 변경한 뒤에도 mAP가 0.986에서 0.979로만 감소해, 성능을 크게 낮추지 않으면서 운영 유연성을 확보했습니다.

04 · LEARNING

배운 점

AI 모델을 설계할 때는 단순 성능 지표뿐 아니라 신규 class 추가, unknown class 대응, 유지보수 같은 운영적 측면도 함께 고려해야 한다는 점을 배웠습니다.

PROCESS

서비스 흐름

01 알약 사진 업로드 › 02 알약 위치 자동 인식 › 03 알약 종류 분석 › 04 분류 결과 확인

2026.03.17 ~ 2026.04.02 · 컴퓨터비전 모델링, 추론 파이프라인 설계, Gradio 기반 API 구현

Python · PyTorch · YOLOv11s · ResNet50 · OpenCV · Roboflow · W&B · GitHub · Gradio

GitHub 저장소 열기 ↗

COMPUTER VISION

딥페이크 이미지 탐지 모델

06

사람 얼굴 이미지를 입력하면 딥페이크 여부를 판별하고, 딥페이크일 확률을 예측하는 컴퓨터비전 기반 AI 모델입니다.



파이프라인	출력 결과	서비스 형태
CNN + ViT	binary + 확률	Docker API

01 · PROBLEM

문제 정의

초기 baseline으로 단일 CNN 모델을 적용했지만 AUC가 0.5에도 미치지 못해, 딥페이크 이미지의 조작 패턴을 충분히 학습하지 못하는 한계가 있었습니다.

02 · APPROACH

해결 방법

관련 논문을 살펴본 뒤 CNN 기반 지역 특징과 ViT 기반 전역 특징을 함께 활용하는 CNN + ViT 구조로 수정하고, 5-class 분류 문제까지 확장했습니다.

03 · OUTCOME

결과

모델 구조와 데이터 처리 방식을 개선한 결과 AUC를 0.5 미만의 baseline에서 0.67까지 끌어올렸습니다.

04 · LEARNING

배운 점

문제 기능별로 효과가 좋은 모델을 적극적으로 서칭하는 것이 중요하며, 데이터 처리 방식 역시 일반화 성능을 높이는 핵심 요소라는 점을 깨달았습니다.

PROCESS

서비스 흐름

01 얼굴 사진 업로드 › 02 이미지 자동 분석 › 03 딥페이크 확률 계산 › 04 판별 결과 확인

2025.12.29 ~ 2026.02.02 · 이미지 분류 모델링, 추론 파이프라인 설계, Docker 기반 API 구현

Python · PyTorch · Xception · ViT · OpenCV · GitHub · Docker · FastAPI

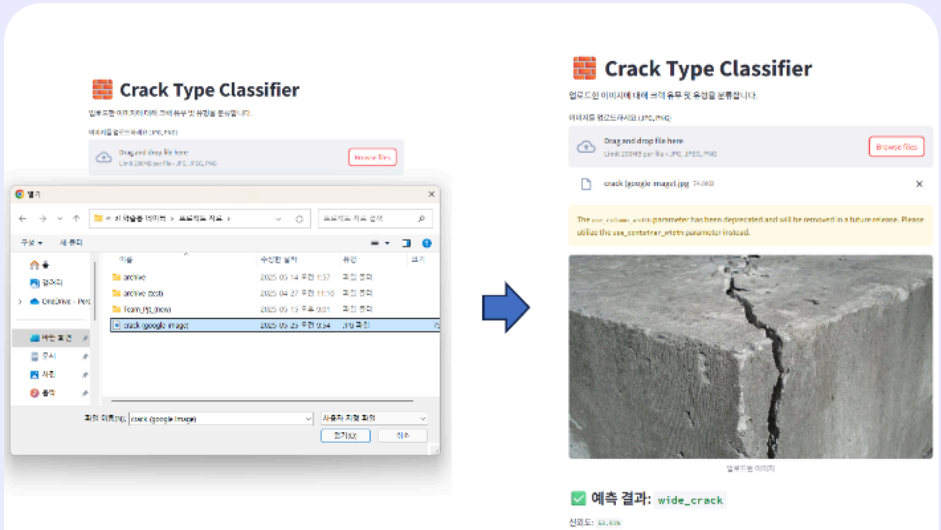
GitHub 저장소 열기 ↗

COMPUTER VISION · TEAM LEAD

콘크리트 균열 탐지 및 유형 분류 모델

07

콘크리트 이미지를 입력하면 MobileNetV2 기반 분류 모델이 crack 없음, wide crack, hairline crack 중 하나로 예측하는 컴퓨터비전 모델입니다.



파이프라인
MobileNetV2 분류

출력 결과
crack 없음 / wide / hairline

서비스 형태
Streamlit

01 · PROBLEM

문제 정의

crack 유형에 대한 라벨이 없는 데이터였기 때문에 crack 없음, wide crack, hairline crack을 바로 분류하기 어려웠고, 초기 분류 방식으로는 충분한 결과를 얻지 못했습니다.

02 · APPROACH

해결 방법

Labelme로 200개 이미지를 수동 segmentation 라벨링하고, 이를 기반으로 U-Net을 학습시켜 나머지 이미지의 균열 영역 라벨을 확장했습니다. 이렇게 구축한 라벨을 활용해 MobileNetV2 기반 3-class 분류 모델을 학습했습니다.

03 · OUTCOME

결과

라벨이 부족한 상황에서도 segmentation을 데이터 구축 단계에 활용해 학습 데이터를 확장했고, 서비스에서는 MobileNetV2가 crack 없음, wide crack, hairline crack을 분류하는 파이프라인을 구성했습니다.

04 · LEARNING

배운 점

문제에 대한 솔루션을 찾을 때 정확도만 보는 것이 아니라, 제한된 시간 안에서 효율적으로 구현 가능한 방법을 모색하는 것이 중요하다는 점을 배웠습니다.

PROCESS

서비스 흐름

01 콘크리트 사진 업로드 › 02 이미지 자동 분석 › 03 균열 유형 분류 › 04 진단 결과 확인

2025.04.26 ~ 2025.05.26 · 팀장, 세그멘테이션 기반 라벨 구축, MobileNetV2 3-class 분류 모델 학습, Streamlit 데모 구현

TensorFlow · Keras · MobileNetV2 · U-Net · Python · OpenCV · Labelme · Streamlit